

第 3 章 确定性推理

一个智能系统不仅应该拥有知识，还应该很好地利用这些知识，即运用知识进行推理和求解问题。智能系统的推理过程实际上是一个思维过程。按推理过程所用知识的确定性，推理可分为确定性推理和不确定性推理。本章重点讨论确定性推理，不确定性推理将在第 5 章中讨论。

3.1 推理概述

3.1.1 推理的概念

推理是人类求解问题的主要思维方法，其任务是利用知识，因而与知识的表达方法有密切的关系。推理是按照某种策略从已有事实和知识推出结论的过程。推理是由程序实现的，称为推理机。在人工智能系统中，推理机利用知识库中的知识，按一定的控制策略求解问题。例如，在医疗诊断专家系统中，知识库存储专家的经验及医学常识，数据库存放病人的症状、化验结果等初始事实，利用该专家系统为病人诊治疾病实际上就是一次推理过程，即从病人的症状及化验结果等初始事实出发，利用知识库中的知识及一定的控制策略，对病情做出诊断，并开出医疗处方。像这样从初始事实出发，不断运用知识库中的已知知识，逐步推出结论的过程就是推理。

3.1.2 推理的分类

人类的智能活动有多种思维方式，人工智能作为对人类智能的模拟，相应地有多种推理方式。下面从不同的角度对推理方式进行分类。

1. 演绎推理、归纳推理、默认推理

推理的基本任务是从一种判断推出另一种判断，如果从推出新判断的途径来分，推理可分为演绎推理、归纳推理和默认推理。

(1) 演绎推理

演绎推理是从全称判断推出特称判断或单称判断的过程，即从一般到个别的推理。演绎推理中最常用的形式是三段论法。三段论由三个判断组成，其中两个判断是前提，分别称为大前提和小前提，另一个判断为结论。例如：

- ❶ 所有的推理系统都是智能系统；
- ❷ 专家系统是推理系统；
- ❸ 所以，专家系统是智能系统。

这就是一个三段论。其中，❶是大前提，描述的是一般知识；❷是小前提，描述的是关于个

体的判断；③是结论，描述的是由大前提推出的适合小前提的新判断。

可以看出，在演绎推理中，结论是蕴含在大前提中的，即从已知判断中推出其中包含的判断，所以演绎推理并没有增加新的知识。

在三段论式的演绎推理中，只要大前提和小前提是正确的，则由它们推出的结论也必然是正确的。

（2）归纳推理

人们对客观事物的认识总是由认识个别事物开始，进而认识事物的普遍规律，其中归纳推理起了重要的作用。归纳推理是从足够多的事例中归纳出一般性结论的推理过程，是一种从个别到一般的推理过程。常用的归纳推理有枚举法和类比法。

枚举法归纳推理是由已观察到的事物都有某属性，而没有观察到相反的事例，从而推出某类事物都有某属性。其推理过程可以形式地表示为：

S_1 是 P

S_2 是 P

...

S_n 是 P

($S_1, S_2, \dots, S_n$ 是 S 类中的个别事物，在枚举中兼容)

S 都是 P

如果从归纳时所选事例的广泛性来划分，枚举法归纳推理又可分为完全归纳推理、不完全归纳推理。完全归纳推理是指在进行归纳时考察了相应事物的全部对象，并根据这些对象是否都具有某种属性，从而推出这个事物是否具有这个属性。不完全归纳推理是指只考察了相应事物的部分对象，就得出了结论。不完全推理得出的结论不具有必然性，属于非必然性推理，而完全归纳推理是必然性推理。由于考察事物的所有对象通常是比较困难的，因而大多数归纳推理是不完全归纳推理。如检查产品质量时，一般是从中随机抽样检查一定比例的产品，如果抽样检查全部合格，就得出产品质量合格的结论，这就是一个不完全归纳推理。

在两个或两类事物的许多属性都相同的基础上，推出它们在其他属性上也相同，这就是类比法归纳推理。类比法归纳可形式化地表示为：

A 具有属性：a, b, c, d, e

B 具有属性：a, b, c, d

B 也具有属性：e

类比法的可靠程度决定于两个或两类事物的相同属性与推出的属性之间的相关程度，相关程度越高，则可靠性越高。

归纳推理是人类思维活动中最基本、最常用的一种推理形式。归纳推理增加了知识，所以在机器学习部分被称为归纳学习，这部分内容将在后面详细介绍。

（3）默认推理

默认推理又称为缺省推理，是在知识不完全的情况下假设某些条件已经具备所进行的推理。例如，在条件 1 已成立的情况下，如果没有足够的证据能证明条件 2 不成立，则默认条件 2 是成立的，并在此默认前提下进行推理，推导出某个结论。由于这种推理允许默认某些条件是成立的，这就摆脱了需要知道全部有关事实才能进行推理的要求，使得在知识不完全的情况下也能进行推理。在默认推理过程中，如果到某一时刻发现原先所做的默认不正确，就要撤销所做的默认以及由此默认推出的所有结论，重新按新情况进行推理。

2. 确定性推理、不确定性推理

如果按推理时所用的知识的确定性来分，推理可分为确定性推理和不确定推理。

如果在推理中所用的知识都是精确的，即可以把知识表示成必然的因果关系，然后进行逻辑推理，推理的结论或者为真，或者为假，这种推理就称为确定性推理（精确推理）。本章介绍的归结反演、基于规则的演绎系统等都是确定性推理。

在人类知识中，有相当一部分属于人们的主观判断，是不精确的和含糊的，由这些知识归纳出来的推理规则往往是不确定的。基于这种不确定的推理规则进行推理，形成的结论也是不确定的，这种推理称为不确定推理（不精确推理）。

在专家系统中主要使用的是不确定推理。

3. 单调推理、非单调推理

如果按推理过程中推出的结论是否单调增加，或者说推出的结论是否越来越接近最终目标来划分，推理又可分为单调推理和非单调推理。

单调推理是指在推理过程中随着推理的向前推进及新知识的加入，推出的结论呈单调增加的趋势，并且越来越接近最终目标。一个演绎推理的逻辑系统有一个无矛盾的公理系统，新加入的结论必须与公理系统兼容，新的结论与已有的知识不发生矛盾，结论总是越来越多，所以演绎推理是单调推理。

非单调推理是指在推理过程中随着推理的向前推进及新的知识的加入，不仅没有加强已推出的结论，反而要否定它，使得推理退回到前面的某一步，重新开始。一般非单调推理是在知识不完全的情况下进行的，由于知识不完全，为了推理进行下去，就要先做某些假设，并在此假设的基础上进行推理，当以后由于新的知识的加入发现原先的假设不正确时，就需要推翻该假设以及由此假设为基础的一切结论，再用新的知识重新进行推理。

人类的推理过程通常是在信息不完全或者情况变化的情况下进行的，所以推理过程往往是非单调的。

4. 启发式推理、非启发式推理

按推理中是否运用与问题有关的启发性知识，推理可分为启发式推理和非启发式推理。

如果在推理过程中，运用与问题有关的启发性知识，即解决问题的策略、技巧及经验，以加快推理过程，提高搜索效率，这种推理过程称为启发式推理。

如果在推理过程中，不运用启发性知识，只按照一般的控制逻辑进行推理，这种推理过程称为非启发式推理。这种方法缺乏对求解问题的针对性，所以推理效率较低，容易出现“组合爆炸”问题。如图搜索策略中的宽度优先搜索法，虽然是完备的算法，但是对于复杂问题的求解将出现“组合爆炸”现象，搜索效率低。

3.1.3 推理的控制策略

推理的控制策略主要是指推理方向的选择、推理所用的搜索策略及冲突解决策略等，一般推理的控制策略与知识表达方法有关，这里仅就产生式系统来介绍推理的控制策略。

1. 推理方向

推理方向用于确定推理的驱动方式。根据推理方向的不同，推理可以分为正向推理、反向推理和正反向混合推理。无论按哪种方式进行推理，一般要求系统具有一个存放知识的

知识库 (Knowledge Base, KB)、一个存放初始事实和中间结果的数据库 (DataBase, DB) 和一个用于推理的推理机。

(1) 正向推理

正向推理是由已知事实出发向结论方向的推理, 也称为事实驱动推理。正向推理的基本思想是: 系统根据用户提供的初始事实, 在知识库中搜索能与之匹配的规则即当前可用的规则, 构成可适用的规则集 (Rule Set, RS), 然后按某种冲突解决策略, 从规则集中选择一条知识进行推理, 并将推出的结论作为中间结果加到数据库中, 成为下一步推理的事实, 再在知识库中选择可适用的知识进行推理; 如此重复, 直到得出最终结论或者知识库中没有可适用的知识为止。正向推理简单、易实现, 但目的性不强, 效率低, 需要用启发性知识解除冲突并控制中间结果的选取, 其中包括必要的回溯。另外, 由于不能反推, 因此系统的解释功能受到影响。

(2) 反向推理

反向推理是以某个假设目标作为出发点的一种推理, 又称为目标驱动推理或逆向推理。反向推理的基本思想是: 首先提出一个假设目标, 然后由此出发, 进一步寻找支持该假设的证据, 若所需的证据都能找到, 则该假设成立, 推理成功; 若无法找到支持该假设的所有证据, 则说明此假设不成立, 需要另作新的假设。与正向推理相比, 反向推理的主要优点是不必使用与目标无关的知识, 目的性强, 同时有利于向用户提供解释。反向推理的缺点是在选择初始目标时具有很大的盲目性, 若假设不正确, 就有可能需要多次提出假设, 影响系统的效率。反向推理比较适合结论单一或直接提出结论要求证实的系统。

(3) 正反向混合推理

前面已介绍, 正向推理效率比较低, 推理过程中可能会推出许多与问题求解无关的子目标; 反向推理中也存在着选择初始假设的盲目性, 同样会降低推理的效率。为解决这些问题, 可把正向推理和反向推理结合起来, 使其各自发挥自己的优势, 取长补短, 像这样正向推理和反向推理相结合的推理方法称为正反向混合推理。

正反向混合推理的一般过程是: 先根据初始事实进行正向推理, 以帮助提出假设, 再用反向推理进一步寻找支持假设的证据; 如此反复, 直到得出结论为止。正反向混合推理集中了正向推理和反向推理的优点, 但其控制策略相对复杂。

2. 搜索策略

推理时要反复用到知识库中的规则, 而知识库中的规则很多, 这样就存在着如何在知识库中寻找可用规则的问题, 即如何确定推理路线, 使其付出的代价尽可能少, 而问题又能得到较好的解决。为了有效地控制规则的选取, 可以采用各种搜索策略。常用的搜索策略有状态空间搜索 (宽度优先搜索、深度优先搜索、有界深度优先搜索等) 和启发式搜索等。本节只简单介绍搜索策略, 具体搜索算法在第 4 章讨论。

3. 冲突解决策略

在推理过程中, 系统要不断地用数据库中的事实与知识库中的规则进行匹配, 当有一个以上规则的条件部分与当前数据库相匹配时, 就需要有一种策略来决定先使用哪一条规则, 这就是冲突解决策略。冲突解决策略实际上就是确定规则的启用顺序。常用的冲突解决策略有下列 7 种。

(1) 专一性排序

如果某条规则的条件部分规定的情况比另一条规则的条件部分所规定的情况更具体, 则

这条规则具有较高的优先级。例如，有如下规则：

规则 1: IF A AND B AND C THEN E

规则 2: IF A AND B AND C AND D THEN F

数据库中的 A 、 B 、 C 、 D 均为真，这时规则 1 和规则 2 都与数据库相匹配，但因为规则 2 的条件部分包括了更多的限制，所以具有较高的优先级。本策略是优先使用针对性较强的产生式规则。

(2) 规则排序

如果规则编排顺序表示了启用的优先级，则称为规则排序。

(3) 数据排序

数据排序是把规则条件部分的所有条件按优先级次序编排起来，在发生冲突时采用，先使用在条件部分包含较高优先级数据的规则。

(4) 就近排序

就近排序是把最近使用的规则放在最优先的位置。如果某条规则经常使用，则倾向于更多地使用这条规则。

(5) 上下文限制

上下文限制是把产生式规则按它们所描述的上下文分组，在某种上下文条件下，只能从与其相对应的那组规则中选择可应用的规则。这样不仅可以减少冲突，由于搜索范围小，也提高了推理的效率。

(6) 按匹配度排序

在不精确匹配中，为了确定两个知识模式是否可以匹配，需要计算这两个模式的相似程度，当其相似度达到某个预先规定的值时，就认为它们是可匹配的。相似度又称为匹配度，除了可用来确定两个知识模式是否可匹配，还可用于冲突解除。若有几条规则均可匹配成功，则可根据它们的匹配度来决定哪一个产生式规则可优先被应用。

(7) 按条件个数排序

如果有多条产生式规则生成的结论相同，则要求条件少的产生式规则被优先应用，因为要求条件少的规则匹配时花费的时间较少。

不同的系统可使用上述这些策略的不同组合，目的是尽量减少冲突的发生，使推理有较快的速度和较高的效率。如何选择冲突解决策略完全是由启发性知识决定的。

3.2 推理的逻辑基础

第 2 章中介绍了谓词逻辑表示法，给出了谓词、个体、命题逻辑、谓词逻辑、谓词公式、谓词公式的解释等，并指出谓词逻辑是一种重要的知识表示方法，是目前为止能够表示人类思维活动规律的一种最精确的形式语言。本节将在此基础上进一步探讨推理的逻辑基础。

3.2.1 谓词公式的永真性和可满足性

1. 谓词公式的永真性

定义 3.1 如果谓词公式 P ，对个体域 D 上的任何一个解释都取得真值 T，则称 P 在 D 上是永真的；如果 P 在每个非空个体域上均永真，则称 P 永真。

定义 3.2 如果谓词公式 P 对于个体域 D 上的所有解释都取得假值 F, 则称 P 在 D 上是永假的; 如果 P 在每个非空个体域上均永假, 则称 P 永假。谓词公式的永假性又称为不可满足性或不相容性。

2. 谓词公式的可满足性

定义 3.3 对于谓词公式 P , 如果至少存在一个解释使得公式 P 在此解释下的真值为 T, 则称公式 P 是可满足的。

按照定义 3.3, 对谓词公式 P , 如果不存在任何解释, 使得 P 的取值为 T, 则称公式 P 是不可满足的。所以, 谓词公式 P 永假与不可满足是等价的。若 P 永假, 则也可称 P 是不可满足的。

3. 谓词公式的等价性与永真蕴含

定义 3.4 设 P 与 Q 是两个谓词公式, D 是它们共同的个体域。若对 D 上的任何一个解释, P 与 Q 的取值都相同, 则公式 P 和 Q 在域 D 上是等价的。如果 D 是任意个体域, 则称 P 和 Q 是等价的, 记作 $P \Leftrightarrow Q$ 。

定义 3.5 对于谓词公式 P 和 Q , 如果 $P \rightarrow Q$ 永真, 则称 P 永真蕴含 Q , 且称 Q 为 P 的逻辑结论, 称 P 为 Q 的前提, 记作 $P \Rightarrow Q$ 。

4. 谓词演算的等价式

(1) 双重否定律 (Double Negation Law)

$$\neg(\neg P(x)) \Leftrightarrow P(x)$$

(2) 德·摩根定律 (L. Mogen)

$$\neg(P(x) \vee Q(x)) \Leftrightarrow \neg P(x) \wedge \neg Q(x)$$

$$\neg(P(x) \wedge Q(x)) \Leftrightarrow \neg P(x) \vee \neg Q(x)$$

(3) 逆否律 (Inverse-negation Law)

$$P(x) \rightarrow Q(x) \Leftrightarrow \neg Q(x) \rightarrow \neg P(x)$$

(4) 分配律 (Assignment Law)

$$P(x) \wedge (Q(x) \vee R(x)) \Leftrightarrow (P(x) \wedge Q(x)) \vee (P(x) \wedge R(x))$$

$$P(x) \vee (Q(x) \wedge R(x)) \Leftrightarrow (P(x) \vee Q(x)) \wedge (P(x) \vee R(x))$$

(5) 结合律 (Association Law)

$$(P(x) \wedge Q(x)) \wedge R(x) \Leftrightarrow P(x) \wedge (Q(x) \wedge R(x))$$

$$(P(x) \vee Q(x)) \vee R(x) \Leftrightarrow P(x) \vee (Q(x) \vee R(x))$$

(6) 蕴含等价式 (Implication Law)

$$P(x) \rightarrow Q(x) \Leftrightarrow \neg P(x) \vee Q(x)$$

(7) 易名规则 (Rename Law)

$$\forall x P(x) \vee \forall x Q(x) \Leftrightarrow \forall x P(x) \vee \forall y Q(y)$$

(8) 量词转换律 (Quantifier Transform Law)

$$\neg \forall x P(x) \Leftrightarrow \exists x \neg P(x)$$

$$\neg \exists x P(x) \Leftrightarrow \forall x \neg P(x)$$

(9) 量词分配律 (Quantifier Assignment Law)

$$\exists x (P(x) \vee Q(x)) \Leftrightarrow \exists x P(x) \vee \exists x Q(x)$$

$$\forall x (P(x) \wedge Q(x)) \Leftrightarrow \forall x P(x) \wedge \forall x Q(x)$$

$$\forall x(P(x) \rightarrow Q(x)) \Leftrightarrow P(x) \rightarrow \forall xQ(x)$$

$$\exists x(P(x) \rightarrow Q(x)) \Leftrightarrow P(x) \rightarrow \exists xQ(x)$$

(10) 量词交换律 (Quantifier Commutative Law)

$$\forall x\forall y(P(x, y)) \Leftrightarrow \forall y\forall x(P(x, y))$$

$$\exists x\exists y(P(x, y)) \Leftrightarrow \exists y\exists x(P(x, y))$$

(11) 量词辖域变换等价式

$$\forall xP(x) \vee Q \Leftrightarrow \forall x(P(x) \vee Q)$$

$$\forall xP(x) \wedge Q \Leftrightarrow \forall x(P(x) \wedge Q)$$

$$\exists xP(x) \vee Q \Leftrightarrow \forall x(P(x) \vee Q)$$

$$\exists xP(x) \wedge Q \Leftrightarrow \exists x(P(x) \wedge Q)$$

其中, Q 不含变量。

(12) 量词消去及引入规则

全称量词消去规则: $\forall xP(x) \Leftrightarrow P(y)$

全称量词引入规则: $P(y) \Leftrightarrow \forall xP(x)$

存在量词消去规则: $\exists xQ(x) \Leftrightarrow Q(c)$ (c 为常量)

存在量词引入规则: $Q(c) \Leftrightarrow \exists xQ(x)$ (c 为常量)

有限域量词消去规则: 设有限个体域为 $D \in d_1, d_2, \dots, d_n$, 则

$$\forall xP(x) \Leftrightarrow P(d_1) \wedge P(d_2), \dots, \wedge P(d_n)$$

$$\exists xQ(x) \Leftrightarrow Q(d_1) \vee Q(d_2), \dots, \vee Q(d_n)$$

5. 推理规则

在推理中, 一些推理规则如下。

❖ **P 规则**: 在推理的任何步骤上都可引入前提。

❖ **T 规则**: 推理时, 如果前面步骤中有一个或多个永真蕴含公式 S , 则可把 S 引入推理过程中。

❖ **CP 规则**: 如果能从 R 和前提集合中推出 S , 则可从前提集合推出 $R \rightarrow S$ 。

❖ **反证法**: $P \Rightarrow Q$, 当且仅当 $P \wedge \neg Q \Leftrightarrow F$, 即 Q 为 P 的逻辑结论, 当且仅当 $P \wedge \neg Q$ 是不可满足的。

推广之, 可得如下定理。

定理 3.1 Q 为 $P_1, P_2, \dots, P_n$ 的逻辑结论, 当且仅当 $(P_1 \wedge P_2 \wedge \dots \wedge P_n) \wedge \neg Q$ 是不可满足的。

3.2.2 置换与合一

置换与合一是为了处理谓词逻辑中的子句之间的模式匹配而引进的。

1. 置换的概念

定义 3.6 形如 $\{t_1/x_1, t_2/x_2, \dots, t_n/x_n\}$ 的一个有限集称为置换, 其中 x_i 是变量, t_i 是不同于 x_i 的项 (常量、变量、函数), 并且要求 t_i 与 x_i 不能相同, x_i 不能循环地出现在另一个 t_i 中 ($i, j=1, 2, \dots, n$)。例如, $\{a/x, b/y, f(x)/z\}$ 、 $\{f(z)/x, y/z\}$ 都是置换, 但 $\{g(y)/x, f(x)/y\}$ 不是一个置换, 原因是它在 x 与 y 之间出现了循环置换现象。通常, 置换用希腊字母 θ 、 σ 、 α 、 λ 等来表示。

不含任何元素的置换称为空置换, 以 ε 表示。

定义 3.7 设 F 为谓词公式, σ 为一个置换, 则称 $F\sigma$ 为 F 的特例, 也称为例或因因子。

置换乘法的作用是将两个置换合成为一个置换。定义如下:

定义 3.8 假设有如下两个置换

$$\begin{aligned}\theta &= \{t_1 / x_1, t_2 / x_2, \dots, t_n / x_n\} \\ \lambda &= \{u_1 / y_1, u_2 / y_2, \dots, u_m / y_m\}\end{aligned}$$

则 θ 与 λ 的合成也是一个置换, 记作 $\theta \square \lambda$, 也称为置换乘法, 其作用于公式 E 时, 相当于先 θ 后 λ 对 E 的作用。它是从集合 $\{t_1 \lambda / x_1, t_2 \lambda / x_2, \dots, t_n \lambda / x_n, u_1 / y_1, u_2 / y_2, \dots, u_m / y_m\}$ 中删除以下两种元素后剩下的元素所构成的集合。

① 当 $t_i \lambda = x_i$ 时, 删除 $t_i \lambda / x_i$ 。

② 当 $y_i \in \{x_1, x_2, \dots, x_n\}$ 时, 删除 u_i / y_i 。

例 3.1 设 $\theta = \{f(y) / x, z / y\}$, $\lambda = \{a / x, b / y, y / z\}$, 求 θ 与 λ 的合成。

解: 先求出集合

$$\{f(b/y)/x, (y/z)/y, a/x, b/y, y/z\} = \{f(b)/x, y/y, a/x, b/y, y/z\}$$

其中, $f(b)/x$ 中的 $f(b)$ 是置换 λ 作用于 $f(y)$ 的结果; y/y 中的 y 是置换 λ 作用于 z 的结果。在该集合中, y/y 满足定义 3.8 中的条件①, 需要删除; a/x 和 b/y 满足定义 3.8 中的条件②, 也需要删除。最后得到

$$\theta \square \lambda = \{f(b)/x, y/z\}$$

这就是 θ 与 λ 的合成。

2. 置换的性质

① 对任意的置换 θ , 恒有 $\varepsilon \square \theta = \theta \square \varepsilon = \theta$ 。

② 对任意的表达式 E , 恒有 $E(\theta \square \lambda) = (E\theta) \square \lambda$ 。

③ 若对任意表达式 E , 恒有 $E\theta = E\lambda$, 则 $\theta = \lambda$ 。

④ 对任意置换 θ 、 λ 、 μ , 恒有 $(\theta \square \lambda) \square \mu = \theta (\lambda \square \mu)$, 即置换的合成满足结合律。注意: 置换的合成不满足交换律。

⑤ 设 A 和 B 为表达式的集合, 则对任意的置换 θ , 恒有 $(A \cup B)\theta = (A\theta) \cup (B\theta)$ 。

3. 合一的概念

定义 3.9 设有公式集 $\{E_1, E_2, \dots, E_n\}$ 和置换 θ , 若 $E_1\theta = E_2\theta = \dots = E_n\theta$ 成立, 则称 $E_1, E_2, \dots, E_n$ 是可合一的, 且 θ 称为合一置换。

例 3.2 设有公式集 $F = \{P(x, y, f(y)), P(a, g(x), z)\}$, 则 $\lambda = \{a/x, g(a)/y, f(g(a))/z\}$ 是它的一个合一。

一般来说, 一个公式集的合一不是唯一的。

定义 3.10 设 σ 为谓词公式 $E_1, E_2, \dots, E_n$ 的一个合一置换, 若对公式 $E_1, E_2, \dots, E_n$ 的任意一个置换 θ , 都存在一个置换 λ , 使得 $\theta = \sigma \square \lambda$, 则称 σ 是 $E_1, E_2, \dots, E_n$ 的最一般合一置换 (Most General Unifier, MGU)。

一个公式集的最一般合一是唯一的。

4. 合一算法 (MGU 算法)

为了描述 MGU 算法, 首先给出差异集 (Difference Set, DS) D 的概念。

定义 3.11 设 W 为一个表达式的非空集合, 则 W 的差异集 D 是按下述方法得出的子表达式的集合。

① 在 W 的所有表达式中找出对应符号不全相同的第一个符号（自左算起）。

② 在 W 的每个表达式中，提取占有该符号位置的子表达式，这些子表达式的集合便是 W 的差异集 D 。

例 3.3 设 $W = \{P(x, f(y, z)), P(x, a), P(x, g(h(k(x))))\}$ ，求 W 的差异集 D 。

解：在 W 的 3 个表达式中，前 4 个符号“ $P(x,$ ”是相同的，第 5 个符号不全相同，所以 W 的差异集 D 为 $D = \{f(y, z), a, g(h(k(x)))\}$ 。假设 D 是 W 的差异集，显然有下面的结论。

(1) 若 D 中无变量符号，则 W 是不可合一的。例如：

$$\begin{aligned} W &= \{P(a), P(b)\} \\ D &= \{a, b\} \end{aligned}$$

(2) 若 D 中只有一个元素，则 W 是不可合一的。例如：

$$\begin{aligned} W &= \{P(x), P(x, y)\} \\ D &= \{y\} \end{aligned}$$

(3) 若 D 中有变量符号 x 和项 t ，且 x 出现在 t 中，则 W 是不可合一的。例如：

$$\begin{aligned} W &= \{P(x), P(f(x))\} \\ D &= \{x, f(x)\} \end{aligned}$$

下面给出 MGU 算法。

算法 3.1 MGU 算法。

① 置 $k=0$ ， $W_k = W$ ， $\sigma_k = \varepsilon$ 。

② 若 W_k 中只有一个元素，则算法停止， σ_k 是要求的 MGU，否则求出 W_k 的差异集 D_k 。

③ 若 D_k 中存在元素 v_k 和 t_k ，并且 v_k 是不出现在 t_k 中的变量，则转向第④步；否则终止，并且 W 是不可合一的。

④ 置 $\sigma_{k+1} = \sigma_k \sqcup \{t_k / x_k\}$ ， $W_{k+1} = W_k \sqcup \{t_k / x_k\}$ （注意 $W_{k+1} = W \sigma_{k+1}$ ）。

⑤ 置 $k=k+1$ ，转向第②步。

注意：在第③步，要求 v_k 不出现在 t_k 中，这称为 Occur 检查，算法的正确性依赖于它。

假如 $W = \{P(x, x), P(y, f(y))\}$ ，执行合一算法。

① $D_0 = \{x, y\}$ 。

② $\sigma_1 = \{y / x\}$ ， $W\sigma_1 = \{P(y, y), P(y, f(y))\}$ 。

③ $D_1 = \{y, f(y)\}$ ，因为 y 出现在 $f(y)$ 中，所以 W 不可合一。若不做 Occur 检查，则算法不能停止。

但是，由于 Occur 检查使上述合一算法在最坏的情况下运行时间是输入长度的指数函数，因此在多数逻辑程序设计语言 Prolog 的实现中都省略了 Occur 检查。

可以证明，若 E_1 和 E_2 可合一，则算法必收敛于第③步。

例 3.4 求 $W = \{P(a, x, f(g(y))), P(z, f(z), f(u))\}$ 的最一般合一。

解：

(1) 令 $W_0 = W$ ， $\sigma_0 = \varepsilon$ 。

(2) W_0 未合一，不一致集为 $D_0 = \{a, z\}$ 。

(3) D_0 中存在变量 $x_0 = z$ 和常量 $t_0 = a$ 。

(4) 令 $\sigma_1 = \sigma_0 \sqcup \{t_0 / x_0\} = \sigma_0 \sqcup \{a / z\}$ ，则：

$$\begin{aligned} \text{①} \quad W_1 &= W_0 \sqcup \{t_0 / x_0\} = \{P(a, x, f(g(y))), P(z, f(z), f(u))\} \{a, z\} \\ &= \{P(a, x, f(g(y))), P(a, f(a), f(u))\} \end{aligned}$$

② W_1 未合一，不一致集为 $D_1 = \{x, f(a)\}$ 。

③ D_1 中存在元素 $x_1 = x$ 和 $t_1 = f(a)$ ，并且变量 x 不出现在 $f(a)$ 中。

④ 令 $\sigma_2 = \sigma_1 \square (t_1 / x_1) = \sigma_1 \square \{f(a) / x\} = \{a / z, f(a) / x\}$ ，则

$$\begin{aligned} W_2 &= W_1 \square (t_1 / x_1) = \{P(a, x, f(g(y))), P(z, f(z), f(u))\} \{a / z, f(a) / x\} \\ &= \{P(a, f(a), f(g(y))), P(a, f(a), f(u))\} \end{aligned}$$

W_2 未合一，不一致集为 $D_2 = \{g(y), u\}$ 。 D_2 中存在元素 $x_2 = u$ 不出现在 $t_2 = g(y)$ 中。

令 $\sigma_3 = \sigma_2 \square (t_2 / x_2) = \sigma_2 \square \{g(y) / u\} = \{a / z, f(a) / x, g(y) / u\}$ ，则

$$\begin{aligned} W_3 &= W_2 \square (t_2 / x_2) = \{P(a, f(a), f(g(y))), P(a, f(a), f(u))\} \{a / z, f(a) / x, g(y) / u\} \\ &= \{P(a, f(a), f(g(y)))\} \end{aligned}$$

W_3 中只含一个元素，所以

$$\sigma_3 = \{P(a, f(a), f(g(y))), P(a, f(a), f(u))\}$$

是 W 的最一般合一，终止。

注意：上述合一算法对任意有限非空的表达式集合总是能够终止的，否则将产生有限非空表达式集合的一个无穷序列 $\sigma_0, W\sigma_1, W\sigma_2, \dots$ ，该序列中的任一集合 $W\sigma_{k+1}$ 都比相应的集合 $W\sigma_k$ 少含一个变量（即 $W\sigma_k$ 含有 v_k ，但 $W\sigma_{k+1}$ 不含 v_k ）。由于 W 中只含有限不同的变量，因此上述情况不会发生。这里不加证明地给出下述定理。

定理 3.2 若 W 为有限非空可合一表达式集合，则合一算法总能终止在例 3.4 的第 (2) 步上，并且最后的 σ_k 便是 W 的最一般合一。

3.3 自然演绎推理

从一组已知为真的事实出发，直接运用经典逻辑中的推理规则推出结论的过程称为自然演绎推理。

自然演绎推理最基本的推理规则是三段论推理，包括：

- ❖ 假言推理： $P, P \rightarrow Q \Rightarrow Q$ 。
- ❖ 拒取式： $\neg Q, P \rightarrow Q \Rightarrow \neg P$ 。
- ❖ 假言三段论： $P \rightarrow Q, Q \rightarrow R \Rightarrow P \rightarrow R$ 。

在自然演绎推理中需要避免两类错误：肯定后件的错误和否定前件的错误。肯定后件的错误是指，当 $P \rightarrow Q$ 为真时，希望通过肯定后件 Q 为真来推出前件 P 为真，这是不允许的。原因是指当 $P \rightarrow Q$ 及 Q 为真时，前件 P 既可能为真，也可能为假。否定前件的错误是指 $P \rightarrow Q$ 为真时，希望通过否定前件 P 来推出后件 Q 为假，这也是不允许的。原因是当 $P \rightarrow Q$ 及 P 为假时，后件 Q 既可能为真，也可能为假。

例 3.5 已知如下事实。

- (1) 只要是需要编写程序的课，王程都喜欢。
- (2) 所有的程序设计语言课都是需要编写程序的课。
- (3) C 是一门程序设计语言课。

求证：王程喜欢 C 这门课。

证明：首先定义谓词。

$\text{Prog}(x)$	x 是需要编写程序的课
$\text{Like}(x, y)$	x 喜欢 y
$\text{Lang}(x)$	x 是一门程序设计语言课

把已知事实及待求解问题用谓词公式表示如下：

$\text{Prog}(x) \rightarrow \text{Like}(\text{Wang}, x)$
 $(\forall x)(\text{Lang}(x) \rightarrow \text{Prog}(x))$
 $\text{Lang}(C)$

应用推理规则进行推理：

$\text{Lang}(y) \rightarrow \text{Prog}(y)$	全称固化
$\text{Lang}(C), \text{Lang}(y) \rightarrow \text{Prog}(y) \Rightarrow \text{Prog}(C)$	假言推理 $\{C/y\}$
$\text{Prog}(C), \text{Prog}(x) \rightarrow \text{Like}(\text{Wang}, x) \Rightarrow \text{Like}(\text{Wang}, C)$	假言推理 $\{C/x\}$

因此，王程喜欢 C 这门课。

一般来说，自然演绎推理由已知事实推出的结论可能有多个，只要包含了需要证明的结论，就认为问题得到了解决。

自然演绎推理定理证明过程自然，易于理解，并且有丰富的推理规则可用。其主要缺点是容易产生知识爆炸，推理过程中得到的中间结论一般按指数规律递增，对于复杂问题的推理不利，甚至难以实现。

3.4 归结演绎推理

3.4.1 子句型

鲁滨逊（Robinson）归结原理是在子句集的基础上讨论问题。因此，讨论归结演绎推理之前需要先讨论子句集的有关概念。

1. 子句和子句集

定义 3.12 原子谓词公式及其否定统称为文字。

例如， $P(x)$ 、 $Q(x)$ 、 $\neg P(x)$ 、 $\neg Q(x)$ 等都是文字。

定义 3.13 任何文字的析取式称为子句。

例如， $P(x) \vee Q(x)$ 、 $P(x, f(x)) \vee Q(x, g(x))$ 都是子句。

定义 3.14 不含任何文字的子句称为空子句。空子句一般被记为 $\square$ 或 NIL。

由于空子句不含有任何文字，也就不能被任何解释所满足，因此空子句是永假的，不可满足的。

定义 3.15 由子句或空子句构成的集合称为子句集。

2. 谓词公式的化简

在谓词逻辑中，任何一个谓词公式都可以通过应用等价关系及推理规则化成相应的子句集。其化简步骤如下：

(1) 消去连接词 “ $\rightarrow$ ” 和 “ $\leftrightarrow$ ”

反复使用如下等价公式：

$$P \rightarrow Q \Leftrightarrow P \vee \neg Q$$

$$P \leftrightarrow Q \Leftrightarrow (P \wedge Q) \vee (\neg P \wedge \neg Q)$$

即可消去谓词公式中的连接词 “ $\rightarrow$ ” 和 “ $\leftrightarrow$ ”。

(2) 减少否定符号的辖域

反复使用双重否定律

$$\neg(\neg P) \Leftrightarrow P$$

摩根定律

$$\neg(P \wedge Q) \Leftrightarrow \neg P \vee \neg Q$$

$$\neg(P \vee Q) \Leftrightarrow \neg P \wedge \neg Q$$

量词转换律

$$\neg(\forall x)P(x) \Leftrightarrow (\exists x)\neg P(x)$$

$$\neg(\exists x)P(x) \Leftrightarrow (\forall x)\neg P(x)$$

将每个否定符号 $\neg$ 移到仅靠谓词的位置，使得每个否定符号最多只作用于一个谓词上。

(3) 对变元标准化

在一个量词的辖域内，把谓词公式中受该量词约束的变元全部用另一个没有出现过的任意变元代替，使不同量词约束的变元有不同的名字。

(4) 化为前束范式

化为前束范式的方法：把所有量词都移到公式的左边，并且在移动时不能改变其相对顺序。由于第(3)步已对变元进行了标准化，每个量词都有自己的变元，这就消除了任何由变元引起冲突的可能，因此这种移动是可行的。

(5) 消去存在量词

消去存在量词时，需要区分以下两种情况。

① 若存在量词不出现在全称量词的辖域内（即它的左边没有全称量词），则只要用一个新的个体常量替换受该存在量词约束的变元，就可消去该存在量词。

② 若存在量词位于一个或多个全称量词的辖域内，如

$$(\forall x_1) \cdots (\forall x_n)(\exists y)P(x_1, x_2, \cdots, x_n, y)$$

则需要用 Skolem 函数 $f(x_1, x_2, \cdots, x_n)$ 替换受该存在量词约束的变元 y ，再消去该存在量词。

(6) 化为 Skolem 标准形

Skolem 标准形的一般形式为

$$(\forall x_1) \cdots (\forall x_n)M(x_1, x_2, \cdots, x_n)$$

其中， $M(x_1, x_2, \cdots, x_n)$ 是 Skolem 标准形的母式，由子句的合取构成。

把谓词公式化为 Skolem 标准形需要使用以下等价关系：

$$P \vee (Q \wedge R) \Leftrightarrow (P \vee Q) \wedge (P \vee R)$$

(7) 消去全称量词

由于母式中的全部变元均受全称量词的约束，并且全称量词的次序已无关紧要，因此可以省掉全称量词，但剩下的母式仍假设其变元是被全称量词量化的。

(8) 消去合取词

在母式中消去所有合取词，把母式用子句集的形式表示出来。其中，子句集中的每个元素都是一个子句。

(9) 更换变量名称

对子句集中的某些变量重新命名，使任意两个子句中不出现相同的变量名。由于每个子句都对应母式中的一个合取元，并且所有变元都是由全称量词量化的，因此任意两个不同子句的变量之间实际上不存在任何关系。这样，更换变量名是不会影响公式的真值的。

例 3.6 化简下列谓词公式：

$$(1) \quad (\forall x)((\forall y)P(x, y) \rightarrow \neg(\forall y)(Q(x, y) \rightarrow R(x, y)))$$

$$(2) \quad \forall x\{P(x) \rightarrow \forall y[P(y) \rightarrow P(f(x, y))] \wedge \neg \forall y[Q(x, y) \rightarrow P(y)]\}$$

解:

对上述谓词公式 (1):

① 消去连接词 “ $\rightarrow$ ”: 经等价变换后, 得到

$$(\forall x)(\neg(\forall y)P(x, y) \vee \neg(\forall y)(\neg Q(x, y) \vee R(x, y)))$$

② 减少否定符号的辖域: 上式经等价变换后为

$$(\forall x)((\exists y)\neg P(x, y) \vee (\exists y)(Q(x, y) \wedge \neg R(x, y)))$$

③ 对变元标准化: 上式经变换后为

$$(\forall x)((\exists y)\neg P(x, y) \vee (\exists z)(Q(x, z) \wedge \neg R(x, z)))$$

④ 化为前束范式: 上式化为前束范式后为

$$(\forall x)((\exists y)(\exists z)\neg P(x, y) \vee (Q(x, z) \wedge \neg R(x, z)))$$

⑤ 消去存在量词: 上式中存在量词 $\exists y$ 和 $\exists z$ 都位于 $\forall x$ 的辖域内, 因此都需要用 Skolem 函数来替换。设替换 y 和 z 的 Skolem 函数分别是 $f(x)$ 和 $g(x)$, 则替换后的式子为

$$(\forall x)(\neg P(x, f(x)) \vee (Q(x, g(x)) \wedge \neg R(x, g(x))))$$

⑥ 化为 Skolem 标准形: 上面的公式化为 Skolem 标准形后为

$$(\forall x)(\neg P(x, f(x)) \vee (Q(x, g(x)) \wedge (\neg P(x, f(x)) \vee \neg R(x, g(x))))$$

⑦ 消去全称量词: 上式消去全称量词后为

$$(\neg P(x, f(x)) \vee Q(x, g(x)) \wedge (\neg P(x, f(x)) \vee \neg R(x, g(x))))$$

⑧ 消去合取词: 上式的子句集中包含以下两个子句

$$\neg P(x, f(x)) \vee Q(x, g(x))$$

$$\neg P(x, f(x)) \vee \neg R(x, g(x))$$

⑨ 更换变量名称: 对上面的两个子句, 可把第二个子句中的变元名 x 更换为 y , 得到如下子句集

$$\neg P(x, f(x)) \vee Q(x, g(x))$$

$$\neg P(y, f(y)) \vee \neg R(y, g(y))$$

对上述谓词公式 (2), 同样可得到:

① 消去蕴含符号: 经等价变换后得到

$$\forall x\{\neg P(x) \vee \forall y[\neg P(y) \wedge P(f(x, y))] \wedge \neg \forall y[Q(x, y) \vee P(y)]\}$$

② 减少否定符号的辖域: 经等价变换后得到

$$\forall x\{\neg P(x) \vee \forall y[\neg P(y) \vee P(f(x, y))] \wedge \exists y[Q(x, y) \wedge \neg P(y)]\}$$

③ 对变元标准化: 重新命名变元名, 使不同量词约束的变元有不同的名字

$$\forall x\{\neg P(x) \vee \forall y[\neg P(y) \vee P(f(x, y))] \wedge \exists w[Q(x, w) \wedge \neg P(w)]\}$$

④ 消去存在量词

$$\forall x\{\neg P(x) \vee \forall y[\neg P(y) \vee P(f(x, y))] \wedge [Q(x, g(x)) \wedge \neg P(g(x))]\}$$

⑤ 化为前束形

$$\forall x \forall y\{\neg P(x) \vee [\neg P(y) \vee P(f(x, y))] \wedge [Q(x, g(x)) \wedge \neg P(g(x))]\}$$

⑥ 把母式化为合取范式

$$\forall x \forall y\{\neg P(x) \vee \neg P(y) \vee P(f(x, y)) \wedge Q(x, g(x)) \wedge \neg P(g(x))\}$$

⑦ 消去全称量词和合取连接词

$$\neg P(x) \vee \neg P(y) \vee P(f(x, y))$$

$$\begin{aligned} & Q(x, g(x)) \\ & \neg P(g(x)) \end{aligned}$$

这就是该公式的子句集。

3. 子句集的应用

在上述化简过程中，由于在消去存在量词时所用的 Skolem 函数可以不同，因此化简后的标准子句集是不唯一的。这样，当原谓词公式为非永假时，它与其标准子句集并不等价。但当原谓词公式为永假（或不可满足）时，其标准子句集则一定是永假的，即 Skolem 化并不影响原谓词公式的永假性。

这个结论很重要，是归结原理的主要依据，可用定理的形式来描述。

定理 3.3 设有谓词公式 F ，其标准子句集为 S ，则 F 为不可满足的充要条件是 S 为不可满足的。

3.4.2 鲁宾逊归结原理

由谓词公式转化为子句集的方法可知，在子句集中子句之间是合取关系。其中，只要有一个子句为不可满足，则整个子句集是不可满足的。另外，前面已经指出空子句是不可满足的。因此，一个子句集中如果包含有空子句，则此子句集一定是不可满足的。

鲁宾逊归结原理是基于上述认识提出来的，其基本思想是：首先把欲证明问题的结论否定，并加入子句集，得到一个扩充的子句集 S' ；然后设法检验子句集 S' 是否含有空子句，若含有空子句，则表明 S' 是不可满足的；若不含有空子句，则继续使用归结法，在子句集中选择合适的子句进行归结，直至导出空子句或不能继续归结为止。鲁宾逊归结原理可分为命题逻辑归结原理和谓词逻辑归结原理。

1. 命题归结逻辑原理

归结推理的核心是求两个子句的归结式，因此需要先讨论归结式的定义和性质，再讨论命题逻辑的归结过程。

(1) 归结式的定义及性质

定义 3.16 若 P 是原子谓词公式，则称 P 与 $\neg P$ 为互补文字。

定义 3.17 设 C_1 和 C_2 是子句集中的任意两个子句，如果 C_1 中的文字 L_1 与 C_2 中的文字 L_2 互补，那么可从 C_1 和 C_2 中分别消去 L_1 和 L_2 ，并将 C_1 和 C_2 中余下的部分按析取关系构成一个新的子句 C_{12} ，则称这一过程为归结，称 C_{12} 为 C_1 和 C_2 的归结式，称 C_1 和 C_2 为 C_{12} 的亲本子句。

例 3.7 设 $C_1 = \neg Q$ ， $C_2 = Q$ ，求 C_1 和 C_2 的归结式 C_{12} 。

解：这里 $L_1 = \neg Q$ ， $L_2 = Q$ ，通过归结可以得到

$$C_{12} = \text{NIL}$$

如果改变归结顺序，那么同样可以得到相同的结果，即其归结过程不是唯一的。

定理 3.4 归结式 C_{12} 是其亲本子句 C_1 和 C_2 的逻辑结论。

上述定理是归结原理中的一个重要定理，由它可得到以下两个推论：

推论 1：设 C_1 和 C_2 是子句集 S 中的两个子句， C_{12} 是 C_1 和 C_2 的归结式，若用 C_{12} 代替 C_1 和 C_2 后得到新的子句集 S_1 ，则由 S_1 的不可满足性可以推出原子句集 S 的不可满足性。即：

$$S_1 \text{ 的不可满足性} \Rightarrow S \text{ 的不可满足性}$$

推论 2: 设 C_1 和 C_2 是子句集 S 中的两个子句, C_{12} 是 C_1 和 C_2 的归结式, 若把 C_{12} 加入 S 中得到新的子句集 S_2 , 则 S 与 S_2 的不可满足性是等价的。即:

$$S_2 \text{ 的不可满足性} \Leftrightarrow S \text{ 的不可满足性}$$

上述两个推论说明, 为证明子句集 S 的不可满足性, 只要对其中可进行归结的子句进行归结, 并把归结式加入子句集 S 中, 或者用归结式代替他的亲本子句, 然后对新的子句集证明其不可满足性就可以了。

如果经归结能得到空子句, 那么根据空子句的不可满足性, 即可得到原子句集 S 是不可满足的结论。

在命题逻辑中, 对不可满足的子句集 S , 其归结原理是完备的。

这种不可满足性可用如下定理描述。

定理 3.5 子句集 S 是不可满足的, 当且仅当存在一个从 S 到空子句的归结过程。

(2) 命题逻辑的归结反演

假设 F 为已知前提, G 为欲证明的结论, 归结原理把证明 G 为 F 的逻辑结论转化为证明 $F \wedge \neg G$ 为不可满足。再根据定理 3.1, 在不可满足的意义上, 公式集 $F \wedge \neg G$ 与其子句集是等价的, 即把公式集上的不可满足转化为子句集上的不可满足。应用归结原理证明定理的过程称为归结反演。

在命题逻辑中, 已知命题 F , 证明命题 G 为真的归结反演过程如下:

- ① 否定目标公式 G , 得 $\neg G$ 。
- ② 把 $\neg G$ 并入到公式集 F 中, 得到 $F \wedge \neg G$ 。
- ③ 把 $F \wedge \neg G$ 化为子句集 S 。
- ④ 应用归结原理对子句集 S 中的子句进行归结, 并把每次得到的归结式并入 S 中。如此反复进行, 若出现空子句, 则停止归结, 此时就证明了 G 为真。

例 3.8 设已知的公式集为 $\{P, (P \wedge Q) \rightarrow R, (S \vee T) \rightarrow Q, T\}$, 求证结论 R 。

解: 假设结论 R 为假, 将 $\neg R$ 加入公式集, 并化为子句集

$$S = \{P, \neg P \wedge \neg Q \vee R, \neg S \vee Q, \neg T \vee Q, T, \neg R\}$$

其归纳演绎树如图 3-1 所示, 由于根部出现空子句, 因此命题得到证明。这个归结证明过程的含义为: 先假设子句集 S 中的所有子句均为真, 即原公式集为真, $\neg R$ 也为真; 然后利用归结原理, 对子句集进行归结, 并把所得的归结式并入子句集中; 重复这一过程, 最后归结出了空子句。根据归结原理的完备性, 可知子句集 S 是不可满足的, 即开始时假设 $\neg R$ 为真是错误的, 这就证明了 R 为真。

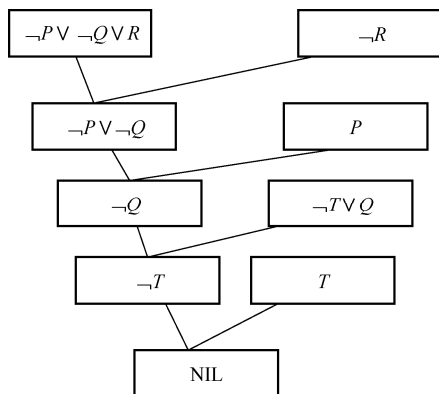


图 3-1 一个命题逻辑的归结演绎

2. 谓词逻辑的归结

在谓词逻辑中, 由于子句集中的谓词一般都含有变元, 因此不能像命题逻辑那样直接消去互补文字, 需要先用一个最一般合一对变元进行代换, 然后才能进行归结。可见, 谓词逻辑的归结要比命题逻辑的归结麻烦一些。

(1) 谓词逻辑的归结原理

谓词逻辑中的归结式可用如下定义来描述。

定义 3.18 设 C_1 和 C_2 是两个没有公共变元的子句, L_1 和 L_2 分别是 C_1 和 C_2 中的文字。如果 L_1 和 L_2 存在最一般合一 σ , 则称

$$C_{12} = (\{C_1\sigma\} - \{L_1\sigma\}) \cup (\{C_2\sigma\} - \{L_2\sigma\})$$

为 C_1 和 C_2 的二元归结式, 而 L_1 和 L_2 为归结式上的文字

例 3.9 设 $C_1 = P(x) \vee Q(a)$, $C_2 = P(b) \vee R(x)$, 求 C_{12} 。

解: 由于 C_1 和 C_2 有相同的变元 x 。为了进行归结, 需要修改 C_2 中变元 x 的名字, 令 $C_2 = \neg P(b) \vee R(y)$ 。此时 $L_1 = P(x)$, $L_2 = \neg P(b)$, L_1 和 L_2 的最一般合一是 $\sigma = \{b/x\}$, 则

$$\begin{aligned} C_{12} &= (\{C_1\sigma\} - \{L_1\sigma\}) \cup (\{C_2\sigma\} - \{L_2\sigma\}) \\ &= (\{P(b) - Q(a)\} - \{P(b)\}) \cup (\{\neg P(b), R(y)\} - \{\neg P(b)\}) \\ &= (\{Q(a)\}) \cup (\{R(y)\}) = \{Q(a), R(y)\} \\ &= Q(a) \vee R(y) \end{aligned}$$

对以上讨论做以下两点说明。

① 这里之所以使用集合符号和集合的运算, 是为了说明问题的方便。即先将子句 $C_i\sigma$ 和 $L_i\sigma$ 写成集合的形式, 在集合表示下做减法和并集运算, 再写成子句集的形式。

② 定义中要求 C_1 和 C_2 无公共变元, 这也是合理的。例如, $C_1 = P(x)$, $C_2 = P(f(x))$, 而 $S = \{C_1, C_2\}$ 是不可满足的。由于 C_1 和 C_2 的变元相同, 就无法合一了。没有归结式, 就不能用归结法证明 S 的不可满足性, 这就限制了归结法的使用范围。

如果对 C_1 或 C_2 的变元进行换名, 便可通过合一, 对 C_1 和 C_2 进行归结。如上例, 若先对 C_2 进行换名, 即 $C_2 = \neg P(f(y))$, 则可对 C_1 和 C_2 进行归结, 得到一个空子句, 从而证明了 S 是不可满足的。

事实上, 在由公式集化为子句集的过程中, 其最后一步就是换名处理。因此, 定义中假设 C_1 和 C_2 没有相同变元是可以的。

例 3.10 设 $C_1 = P(x) \vee P(f(a)) \vee Q(x)$, $C_2 = \neg P(y) \vee R(b)$, 求 C_{12} 。

解: 对参加归结的某个子句, 若其内部有可合一的文字, 则在进行归结之前应先对这些文字进行合一。本例的 C_1 中有可合一的文字 $P(x)$ 与 $P(f(a))$, 若用它们的最一般合一 $\sigma = \{f(a)/x\}$ 进行代换, 可得到

$$C_1\sigma = P(f(a)) \vee Q(f(a))$$

此时可对 $C_1\sigma$ 与 C_2 进行归结。选定 $L_1 = P(f(a))$, $L_2 = \neg P(y)$, L_1 和 L_2 的最一般合一是 $\sigma = \{f(a)/y\}$, 则可得到 C_1 和 C_2 的二元归结式为

$$C_{12} = R(b) \vee Q(f(a))$$

我们把 $C_1\sigma$ 称为 C_1 的因子。一般来说, 若子句 C 中有两个或两个以上的文字具有最一般合一 σ , 则称 $C\sigma$ 为子句 C 的因子。如果 $C\sigma$ 是一个单文字, 则称它为 C 的单元因子。

应用因子概念, 可对谓词逻辑中的归结原理给出如下定义。

定义 3.19 若 C_1 和 C_2 是无公共变元的子句, 则

- ① C_1 和 C_2 的二元归结式。
- ② C_1 和 C_2 的因子 $C_2\sigma_2$ 的二元归结式。
- ③ C_1 的因子 $C_1\sigma_1$ 和 C_2 的二元归结式。
- ④ C_1 的因子 $C_1\sigma_1$ 和 C_2 的因子 $C_2\sigma_2$ 的二元归结式。

这 4 种二元归结式都是子句 C_1 和 C_2 的二元归结式，记为 C_{12} 。

(2) 谓词逻辑的归结反演

对谓词逻辑，定理 3.4 仍然适用，即归结式 C_{12} 是其亲本子句 C_1 和 C_2 的逻辑结论。用归结式取代它在子句集 S 中的亲本子句，得到的子句集仍然保持着原子句集 S 的不可满足性。

此外，定理 3.5 对谓词逻辑仍然适用，即从不可满足的意义上说，一阶谓词逻辑的归结原理也是完备的。

谓词逻辑的归结反演过程与命题逻辑的归结反演过程相比，其步骤基本相同，但每步的处理对象不同。例如，在步骤③化简子句集时，谓词逻辑需要把由谓词构成的公式集化为子句集；在步骤④按归结原理进行归结时，谓词逻辑的归结原理需要考虑两个亲本子句的最一般合一。

谓词逻辑的归结反演步骤如下：设被证明的定理可用谓词公式表示为如下形式

$$A_1 \wedge A_2 \wedge \cdots \wedge A_n \rightarrow B$$

- ① 否定结论 B ，并将否定后的公式 $\neg B$ 与前提公式集组成如下形式的谓词公式：

$$G = A_1 \wedge A_2 \wedge \cdots \wedge A_n \wedge \neg B$$

- ② 求谓词公式 G 的子句集 S 。

- ③ 应用归结原理，证明子句集 S 的不可满足性，从而证明谓词公式 G 的不可满足性。这就说明对结论 B 的否定是错误的，推断出定理的成立。

例 3.11 已知

$$F : (\forall x)((\exists y)(A(x, y) \wedge B(y)) \rightarrow (\exists y)(C(y) \wedge D(x, y)))$$

$$G : \neg(\exists x)C(x) \rightarrow (\forall x)(\forall y)(A(x, y) \rightarrow \neg B(y))$$

求证： G 是 F 的逻辑结论。

证明：先把 G 否定，并放入 F 中，得到的 $\{F, \neg G\}$ 为

$$\{(\forall x)((\exists y)(A(x, y) \wedge B(y)) \rightarrow (\exists y)(C(y) \wedge D(x, y))), \\ \neg(\neg(\exists x)C(x) \rightarrow (\forall x)(\forall y)A(x, y) \rightarrow \neg B(y))\}$$

再把 $\{F, \neg G\}$ 化成子句集，得到

- ① $\neg A(x, y) \vee \neg B(y) \vee C(f(x))$
- ② $\neg A(u, v) \vee \neg B(v) \vee D(u, f(u))$
- ③ $\neg C(z)$
- ④ $A(m, n)$
- ⑤ $B(k)$

其中，①、②是由 F 化出的两个子句，③、④、⑤是由 $\neg G$ 化出的 3 个子句。

最后应用谓词逻辑的归结原理对上述子句集进行归结，其过程为：

- ⑥ $\neg A(x, y) \vee \neg B(y)$ 由 ① 和 ③ 归结，取 $\sigma = \{f(x)/z\}$
- ⑦ $\neg B(n)$ 由 ④ 和 ⑥ 归结，取 $\sigma = \{m/x, n/y\}$
- ⑧ NIL 由 ⑤ 和 ⑦ 归结，取 $\sigma = \{n/k\}$

因此， G 是 F 的逻辑结论。上述归结过程可用如图 3-2 所示的归结树来表示。为了进一步加深对谓词逻辑归结的理解，下面再给出一个经典的归结问题。

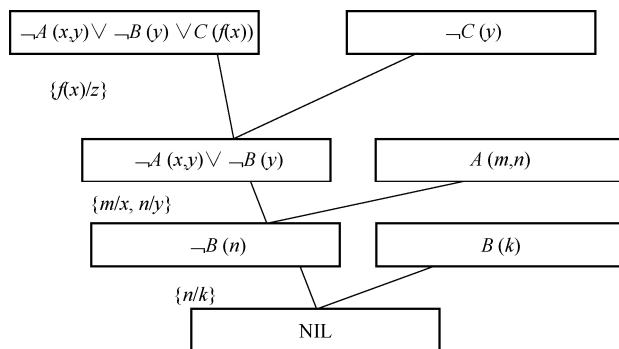


图 3-2 例 3.11 的归结树

例 3.12 “快乐学生”问题。

假设：任何通过计算机考试并获奖的人都是快乐的，任何肯学习或幸运的人都可以通过所有考试，张不肯学习但他是幸运的，任何幸运的人都能获奖。

求证：张是快乐的。

证明：先定义谓词：

$\text{Pass}(x, y)$	x 可以通过 y 考试
$\text{Win}(x, \text{prize})$	x 能获得奖励
$\text{Study}(x)$	x 肯学习
$\text{Happy}(x)$	x 是快乐的
$\text{Lucky}(x)$	x 是幸运的

将上述谓词公式转化为子句集如下：

- (1) $\neg \text{Pass}(x, \text{computer}) \vee \neg \text{Win}(x, \text{prize}) \vee \text{Happy}(x)$
- (2) $\neg \text{Study}(y) \vee \text{Pass}(y, z)$
- (3) $\neg \text{Lucky}(u) \vee \text{Pass}(u, v)$
- (4) $\neg \text{Study}(\text{zhang})$
- (5) $\text{Lucky}(\text{zhang})$
- (6) $\neg \text{Lucky}(w) \vee \text{Win}(w, \text{prize})$
- (7) $\neg \text{Happy}(\text{zhang})$ 结论的否定
- (8) $\neg \text{Pass}(w, \text{Computer}) \vee \text{Happy}(w) \vee \neg \text{Lucky}(w)$ 由 (1) 和 (6) 归结, 取 $\sigma = \{w/x\}$
- (9) $\neg \text{Pass}(\text{zhang}, \text{Computer}) \vee \neg \text{Lucky}(\text{zhang})$ 由 (7) 和 (8) 归结, 取 $\sigma = \{\text{zhang}/w\}$
- (10) $\neg \text{Pass}(\text{zhang}, \text{Computer})$ 由 (5) 和 (9) 归结
- (11) $\neg \text{Lucky}(\text{zhang})$ 由 (3) 和 (10) 归结, 取 $\sigma = \{\text{zhang}/u, \text{computer}/v\}$
- (12) NIL 由 (5) 和 (11) 归结

同样，上述归结过程可用如图 3-3 所示的归结树来表示。

3.4.3 归结演绎推理的归结策略

归结演绎推理实际上是从子句集中不断寻找可进行归结的子句对，并通过对这些子句对的归结，最终得出一个空子句的过程。由于事先并不知道哪些子句对可进行归结，更不知道通过对哪些子句对的归结能尽快得到空子句，因此需要对子句集中的所有子句逐对进行比较，直到得出空子句为止。这种盲目的全面进行归结的方法不仅会产生许多无用的归结式，更严重的是会产生组合爆炸问题，因此需要研究有效的归结策略来解决这些问题。

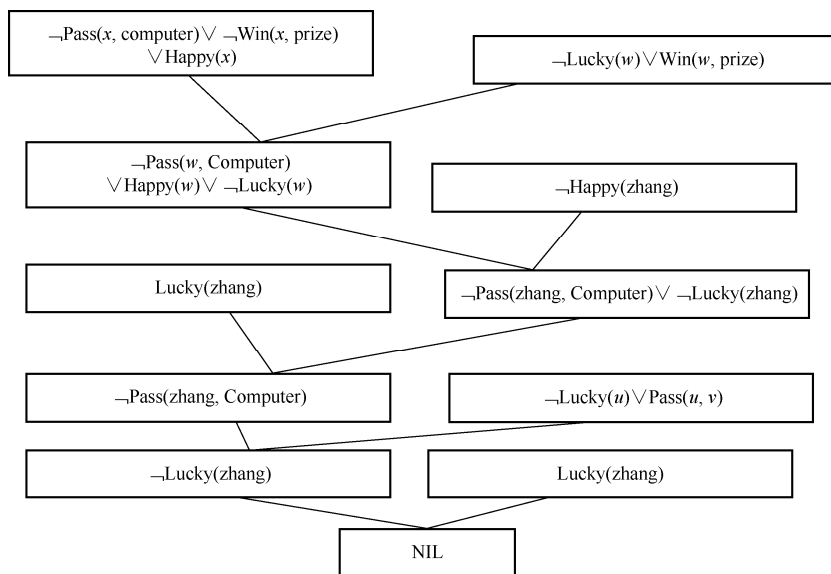


图 3-3 例 3.12 的归结树

目前，常用的归结策略可分为两大类：删除策略、限制策略。删除策略是通过删除某些无用的子句来缩小归结范围；限制策略是通过对参加归结的子句进行某些限制，来减少归结的盲目性，以尽快得到空子句。为了说明选择归结策略的重要性，在讨论各种常用的归结策略之前，下面先介绍宽度优先策略。

1. 宽度优先搜索

宽度优先是一种穷尽子句比较的复杂搜索方法。设初始子句集为 S_0 ，宽度优先策略的归结过程可描述如下。

① 从 S_0 出发，对 S_0 中的全部子句进行所有可能的归结，得到第一层归结式，把这些归结式的集合记为 S_1 。

② 用 S_0 中的子句与 S_1 中的子句进行所有可能的归结，得到第二层归结式，把这些归结式的集合记为 S_2 。

③ 用 S_0 和 S_1 中的子句与 S_2 中的子句进行所有可能的归结，得到第三层归结式，把这些归结式的集合记为 S_3 。

④ 如此继续，直到得出空子句或不能再继续归结为止。

例 3.13 设有如下子句集：

$$S = \{\neg I(x) \vee R(x), I(a), \neg R(y) \vee L(y), \neg L(a)\}$$

用宽度优先策略证明 S 为不可满足。

证明：从初始子句集 S 出发，依次构造 $S_1, S_2, \dots$ ，直到出现空子句结束。宽度优先策略的归结树如图 3-4 所示。

可以看出，宽度优先策略归结出了许多无用的子句，既浪费时间，又浪费空间。但是这种策略有一个有趣的特性，就是当问题有解时确保找到最短归结路径。因此，宽度优先策略是一种完备的归结策略。宽度优先对大问题的归结容易产生组合爆炸，对小问题的归结仍是一种比较好的归结策略。

2. 支持集策略

支持集策略是沃斯（Wos）等人在 1965 年提出的一种归结策略，要求每次参加归结的

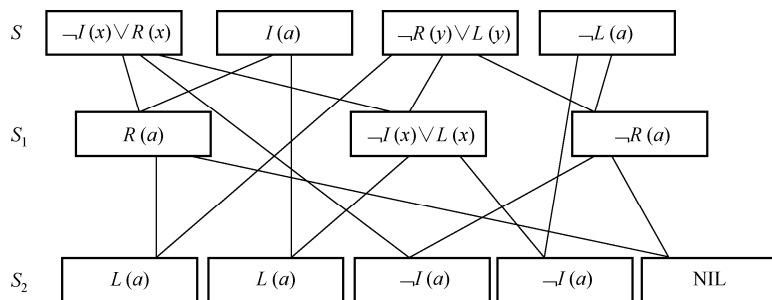


图 3-4 宽度优先策略的归结树

两个亲本子句中至少应该有一个是由目标公式的否定所得到的子句或它们的后裔。可以证明，支持集策略是完备的，即当子句集为不可满足时，则由支持集策略一定能够归结出一个空子句。也可以把支持集策略看成在宽度优先策略中引入了某种限制条件，这种限制条件代表一种启发信息，因而有较高的效率。

例 3.14 设有如下子句集：

$$S = \{\neg I(x) \vee R(x), I(a), \neg R(y) \vee L(y), \neg L(a)\}$$

其中， $\neg I(x) \vee R(x)$ 为目标公式的否定。用支持集策略证明 S 为不可满足。

证明：从 S 出发，其归结树如图 3-5 所示。

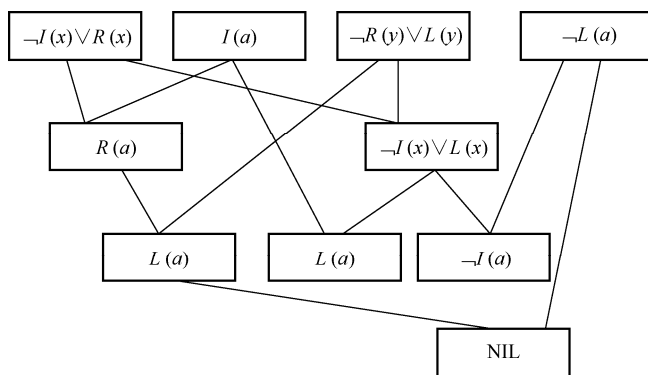


图 3-5 支持集策略的归结树

从上述归结过程可以看出，各级归结式的数目比宽度优先策略生成的少，但在第二级还没有空子句。也就是说，这种策略限制了子句集元素的剧增，但会增加空子句所在的深度。此外，支持集策略具有逆向推理的含义，由于进行归结的亲本子句中至少有一个与目标子句有关，因此推理过程可以看成沿目标、子目标的方向前进。

3. 删除策略

删除策略的主要思想是：归结过程在寻找可归结子句时，子句集中的子句越多，需要付出的代价越大。如果在归结时能把子句集中无用的子句删除掉，就会缩小搜索范围，减少比较次数，从而提高归结效率。常用的删除方法有以下几种。

(1) 纯文字删除法

如果文字 L 在子句集中不存在可与其互补的文字 $\neg L$ ，则称该文字为纯文字。

在归结过程中，纯文字不可能被消除，用包含纯文字的子句进行归结也不可能得到空子句，因此对包含纯文字的子句进行归结是没有意义的，应该把它从子句集中删除。

对于子句集而言，删除包含纯文字的子句是不影响其不可满足性的。例如，有子句集

$$S = \{P \vee Q \vee R, \neg Q \vee R, Q, \neg R\}$$

其中, P 是纯文字, 因此可以将子句 $P \vee Q \vee R$ 从子句集 S 中删除。

(2) 重言式删除法

如果一个子句中包含有互补的文字对, 则称该子句为重言式。例如:

$$\begin{aligned} P(x) \vee \neg P(x) \\ P(x) \vee Q(x) \vee \neg P(x) \end{aligned}$$

都是重言式, 不管 $P(x)$ 的真值为真还是为假, $P(x) \vee \neg P(x)$ 和 $P(x) \vee Q(x) \vee \neg P(x)$ 均为真。

重言式是真值为真的子句。对一个子句集来说, 不管是增加还是删除一个真值为真的子句, 都不会影响该子句集的不可满足性。因此, 可从子句集中删去重言式。

(3) 包孕删除法

设有子句 C_1 和 C_2 , 如果存在一个置换 σ , 使得 $C_1\sigma \subseteq C_2$, 则称 C_1 包孕于 C_2 。例如:

$P(x)$	包孕于 $P(y) \vee Q(z)$	$\sigma = \{x / y\}$
$P(x)$	包孕于 $P(a)$	$\sigma = \{a / x\}$
$P(x)$	包孕于 $P(a) \vee Q(z)$	$\sigma = \{a / x\}$
$P(x) \vee Q(a)$	包孕于 $P(f(a)) \vee Q(a) \vee R(y)$	$\sigma = \{f(a) / x\}$
$P(x) \vee Q(y)$	包孕于 $P(a) \vee Q(u) \vee R(w)$	$\sigma = \{a / x, u / y\}$

对子句集来说, 把其中包孕的子句删去, 不会影响该子句集的不可满足性。因此, 可从子句集中删除包孕的子句。

4. 单文字子句策略

如果一个子句只包含一个文字, 则称此子句为单文字子句。单文字子句策略是对支持集策略的进一步改进, 它要求每次参加归结的两个亲本子句中至少有一个子句是单文字子句。

例 3.15 设有如下子句集:

$$S = \{\neg I(x) \vee R(x), I(a), \neg R(y) \vee L(y), \neg L(a)\}$$

用单文字子句策略证明 S 为不可满足。

证明: 从 S 出发, 其归结树如图 3-6 所示。

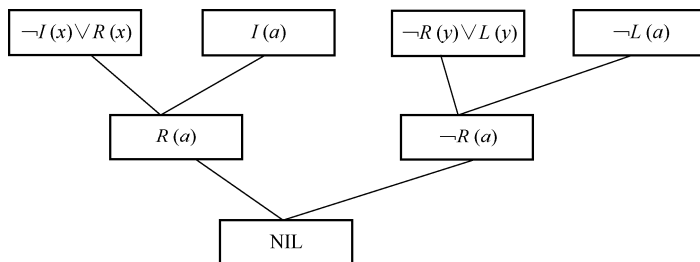


图 3-6 单文字子句策略归结树

采用单文字子句策略, 归结式包含的文字数将少于其亲本子句中的文字数, 这有利于向空子句的方向发展, 因此会有较高的归结效率。但这种策略是不完备的, 即当子句集为不可满足时, 用这种策略不一定能归结出空子句。

5. 线性输入策略

线性输入策略要求每次参加归结的两个亲本子句中, 至少应该有一个是初始子句集中的子句。所谓初始子句集, 是指开始归结时所使用的子句集。

例 3.16 设有子句集:

$$S = \{\neg I(x) \vee R(x), I(a), \neg R(y) \vee L(y), \neg L(a)\}$$

用线性输入策略证明 S 为不可满足。

证明：从 S 出发，其归结树如图 3-7 所示。

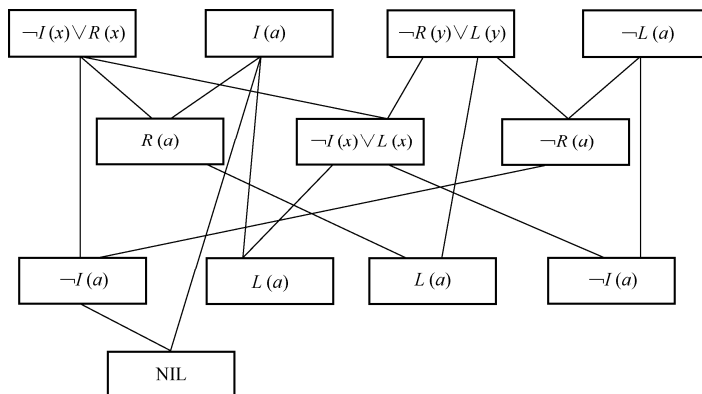


图 3-7 线性输入策略的归结树

线性输入策略可限制生成归结式的数目，具有简单和高效的优点。但是，这种策略也是一种不完备的策略。例如，子句集

$$S = \{Q(u) \vee P(a), \neg Q(w) \vee P(w), \neg Q(x) \vee \neg P(x), Q(y) \vee \neg P(y)\}$$

从 S 出发容易找到一棵归结反演树，却不在线性输入策略的归结反演树。

6. 祖先过滤策略

祖先过滤策略与线性输入策略有点相似，但是放宽了对子句的限制。每次参加归结的两个亲本子句，只要满足以下两个条件中的任意一个就可进行归结：

① 两个亲本子句中至少有一个是初始子句集中的子句。

② 如果两个亲本子句都不是初始子句集中的子句，则一个子句应该是另一个子句的先辈子句。所谓一个子句（如 C_1 ）是另一个子句（如 C_2 ）的先辈子句，是指 C_2 是由 C_1 与其他子句归结后得到的归结式。

例 3.17 有如下子句集：

$$S = \{\neg Q(x) \vee \neg P(x), Q(y) \vee \neg P(y), \neg Q(w) \vee \neg P(w), Q(a) \vee P(a)\}$$

用祖先过滤策略证明 S 为不可满足。

证明：从 S 出发，其归结树如图 3-8 所示。

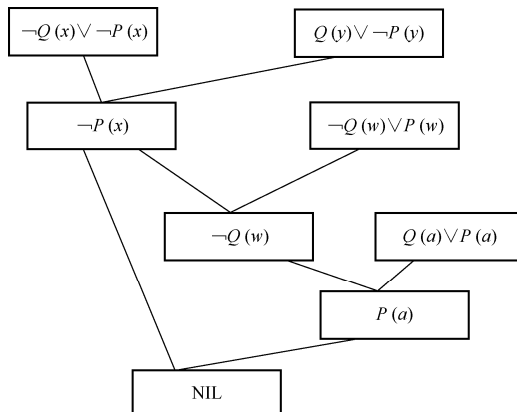


图 3-8 祖先过滤策略的归结树

可以证明祖先过滤策略也是完备的。

上面分别讨论了几种基本的归结策略，在实际应用中可以把几种策略结合起来使用。总之，在选择归结反演策略时，主要应考虑其完备性和效率问题。

3.4.4 用归结原理求取问题的答案

归结原理可用于定理证明，还可用来求取问题答案，其思想与定理证明相似。其一般步骤如下。

<1> 把问题的已知条件用谓词公式表示出来，并化为相应的子句集。

<2> 把问题的目标的否定用谓词公式表示出来，并化为子句集。

<3> 对目标否定子句集中的每个子句，构造该子句的重言式（即把该目标否定子句和此目标否定子句的否定之间再进行析取所得到的子句），用这些重言式代替相应的目标否定子句式，并把这些重言式加入前提子句集中，得到一个新的子句集。

<4> 对这个新的子句集，应用归结原理求出其证明树，这时证明树的根子句不为空，称这个证明树为修改的证明树。

<5> 用修改证明树的根子句作为回答语句，则答案就在此根子句中。

下面通过一个例子来说明如何求取问题的答案。

例 3.18 已知“张和李是同班同学，如果 x 和 y 是同班同学，则 x 的教室也是 y 的教室，现在张在 302 教室上课。”问：“现在李在哪个教室上课？”

解：首先定义谓词：

$C(x, y)$ x 和 y 是同班同学

$At(x, u)$ x 在 u 教室上课

把已知前提用谓词公式表示如下：

$C(\text{zhang}, \text{li})$

$(\forall x)(\forall y)(\forall u)(C(x, y) \wedge At(x, u) \rightarrow At(y, u))$

$At(\text{zhang}, 302)$

把目标的否定用谓词公式表示如下：

$\neg(\exists v)At(\text{li}, v)$

把上述公式化为子句集：

$C(\text{zhang}, \text{li})$

$\neg C(x, y) \vee \neg At(x, u) \vee At(y, u)$

$At(\text{zhang}, 302)$

把目标的否定化成子句式，并用重言式

$\neg At(\text{li}, u) \vee At(\text{li}, u)$

代替之。

把此重言式加入前提子句集中，得到一个新的子句集，对这个新的子句集，应用归结原理求出其证明树，即“李明在 302 教室”。

小 结

推理是智能行为的基本特征之一，理所当然成为人工智能的核心问题之一。本章主要介绍了推理的基本概念和推理的逻辑基础、自然演绎推理和归结演绎推理。

推理是人工智能中的一个非常重要的问题,为了让计算机具有智能,必须使它能够进行推理。所谓推理,就是根据一定的原则,从已知的判断得出另一个新判断的思维过程,是对人类思维的模拟。

人类有多种思维方式,相应地,人工智能中也有多种推理方式。其中,演绎推理和归纳推理是使用较多的两种。演绎推理是由一组前提必然地推出某个结论的过程,是由一般到个别的推理,常用的推理形式是三段论,目前在知识系统中主要采用演绎推理。归纳推理是从足够的事例中归纳出一般性知识的过程,是由个别到一般的推理。

按逻辑规则进行的推理称为逻辑推理。由于逻辑有经典逻辑与非经典逻辑之分,因而逻辑推理也分为经典逻辑、非经典逻辑推理两大类。经典逻辑主要是指命题逻辑与一阶谓词逻辑,由于其值只有“真”与“假”,因而经典逻辑推理中的已知事实以及推出的结论都是精确的,或者为“真”,或者为“假”,所以又称经典逻辑为精确推理或确定性推理。非经典逻辑推理是指除经典逻辑外的那些逻辑,如多值逻辑、模糊逻辑、概率逻辑等,基于这些逻辑的推理称为非经典逻辑推理,是一种不确定性推理。

经典逻辑推理是通过运用经典逻辑规则,从已知事实中演绎出逻辑上蕴含的结论的过程。本章介绍了自然演绎推理,并重点介绍了归结演绎推理。归结演绎推理的理论基础是海伯伦(Herbrand)定理、鲁滨逊归结原理。

习 题 3

3.1 什么是推理、正向推理、反向推理?试列出常用的几种推理方式,并列出每种推理方式的特点。

3.2 在产生式系统中解决冲突的策略有哪些?

3.3 请写出利用归结原理求问题答案的步骤。

3.4 化下列逻辑表达式为不含存在量词的前束形:

$$(\exists x)(\forall y)[(\forall z)P(x,z) \rightarrow R(x,y,f(a))]$$

3.5 判断下列公式是否为可合一,若可合一,则求出其最一般合一 MGU。

(1) $P(a,b), P(x,y)$

(2) $P(f(x),b), P(y,z)$

(3) $P(f(x),y), P(y,f(b))$

(4) $P(f(y),y,x), P(x,f(a),f(b))$

(5) $P(x,y), P(y,x)$

3.6 设已知:

(1) 如果 x 是 y 的父亲, y 是 z 的父亲, 则 x 是 z 的祖父;

(2) 每个人都有一个父亲。

使用归结演绎推理证明: 对于某人 u , 一定存在一个人 v , v 是 u 的祖父。

3.7 判断下列子句集中哪些是不可满足的:

(1) $\{ \neg P \vee Q, \neg Q, P, \neg P \}$

(2) $\{ P \vee Q, \neg P \vee Q, P \vee \neg Q, \neg P \vee \neg Q \}$

(3) $\{ P(y) \vee Q(y), \neg P(f(x)) \vee R(a) \}$

(4) $\{ \neg P(x) \vee Q(x), \neg P(y) \vee R(y), P(a), S(a), \neg S(z) \vee \neg R(z) \}$

(5) $\{ \neg P(x) \vee Q(f(x)), \neg P(h(y)) \vee R(f(h(y))), a \vee \neg P(z) \}$

(6) $\{P(x) \vee Q(x) \vee R(x), \neg P(y) \vee R(y), \neg Q(a), \neg R(b)\}$

3.8 把下列谓词公式分别化为相应的子句集。

(1) $(\forall x)(\forall y)(P(x, y) \rightarrow Q(x, y))$

(2) $(\forall x)(\exists y)(P(x, y) \vee Q(x, y) \rightarrow R(x, y))$

(3) $(\forall x)\{(\forall y)P(x, y) \rightarrow \neg(\forall y)(Q(x, y) \rightarrow R(x, y))\}$

(4) $(\forall x)((\exists y)A(x, y) \wedge B(y)) \rightarrow (\exists y)(C(y) \wedge D(x, y))$

(5) $\neg(\exists x)C(x) \rightarrow (\forall x)(\forall y)(A(x, y) \rightarrow \neg B(y))$

3.9 用归结反演法证明下列公式的永真性。

(1) $(\exists x)\{P(x) \rightarrow P(A)\} \wedge (P(x) \rightarrow P(B))\}$

(2) $(\forall x)\{P(x) \wedge (Q(A) \vee Q(B))\} \rightarrow (\exists x)(P(x) \rightarrow Q(x))$

3.10 对下列各题分别判断 G 是否为 $F_1, F_2, \dots, F_n$ 的逻辑结论, 若是, 则加以证明。

(1) $F: (x)(y)P(x, y)$

$G: (y)(x)P(x, y)$

(2) $F: (x)(P(x) \wedge (Q(a) \vee Q(b)))$

$G: (x)(P(x) \wedge Q(x))$

(3) $F: (x)(y)(P(f(x)) \wedge (Q(f(y))))$

$G: P(f(a)) \wedge P(y) \wedge Q(y)$

(4) $F_1: (x)(P(x) \rightarrow (y)(Q(y) \rightarrow L(x, y)))$

$F_2: (x)(P(x) \wedge (y)(R(y) \rightarrow L(x, y)))$

$G: (x)(R(x) \rightarrow (y)Q(x))$

(5) $F_1: (x)(P(x) \rightarrow ((Q(x) \wedge R(x)))$

$F_2: (x)(P(x) \wedge S(x))$

$G: (x)(S(x) \wedge R(x))$

(6) $F_1: (z)(A(z) \wedge B(z) \rightarrow (y)(D(z, y) \wedge C(y)))$

$F_2: (z)(E(z) \wedge A(z) \wedge (y)(D(z, y) \rightarrow E(y)))$

$F_3: (z)(E(z) \rightarrow B(z))$

$G: (z)(E(z) \rightarrow C(z))$

3.11 “激动人心的生活”问题。

假设: 所有不贫穷并且聪明的人都是快乐的, 那些看书的人是聪明的; 李明能看书且不贫穷, 快乐的人过着激动人心的生活。

求证: 李明过着激动人心的生活。

3.12 已知: 能阅读的人是识字的; 海豚不识字; 有些海豚是很聪明的。

请用归结演绎推理证明: 有些很聪明的人并不识字。

3.13 假设张被盗, 公安局派出 5 个人去调查。案情分析时, 调查员 A 说: “赵与钱中至少有一个人作案”, 调查员 B 说: “钱与孙中至少有一个人作案”, 调查员 C 说: “孙与李中至少有一个人作案”, 调查员 D 说: “赵与孙中至少有一人与此案无关”, 调查员 E 说: “钱与李中至少有一人与此案无关”。如果这 5 个调查员的话都是可信的, 请使用归结演绎推理求出谁是盗窃犯。

3.14 设有子句集:

$$\{P(x) \vee Q(a, b), P(a) \vee Q(a, b), Q(a, f(a)), P(x) \vee Q(x, b)\}$$

分别用各种归结策略求出其归结式。

3.15 对子句集:

$$\{P \vee Q, Q \vee R, R \vee W, R \vee P, W \vee Q, Q \vee R\}$$

用线性输入策略是否可证明该子句集的不可满足性？

3.16 设已知事实为

$$((P \vee Q) \wedge R \vee (S \wedge (T \vee U)))$$

F 规则为

$$S \rightarrow (X \wedge Y) \vee Z$$

试用正向演绎推理推出所有可能的子目标。

3.17 设有如下一段知识：

“小张、小王和小吴都属于高山协会。该协会的每个成员不是滑雪运动员，就是登山运动员，其中不喜欢雨的运动员是登山运动员，不喜欢雪的运动员不是滑雪运动员。

小王不喜欢小张所喜欢的一切东西，而喜欢小张所不喜欢的一切东西。小张喜欢雨和雪。”

试用谓词公式集合表示这段知识，这些谓词公式要适合一个逆向的基于规则的演绎系统。试说明这样一个系统怎样才能回答问题：“高山俱乐部中有没有一个成员，他是一个登山运动员，但不是一个滑雪运动员。”